

Lecture 3: Neural Networks

Today:

00:00) Revisit feature transform

00:10) What is a neural net?

00:20) Chain-rule reminder

00:30) Vanilla back-propagation

00:45) ---- Break ----

01:00) Vanilla back-propagation (continued)

02:30) Differential computational graph

01:45) ---- Break ----

02:00) Differential computational graph (continued)

02:10) Can anything be back-propagated?

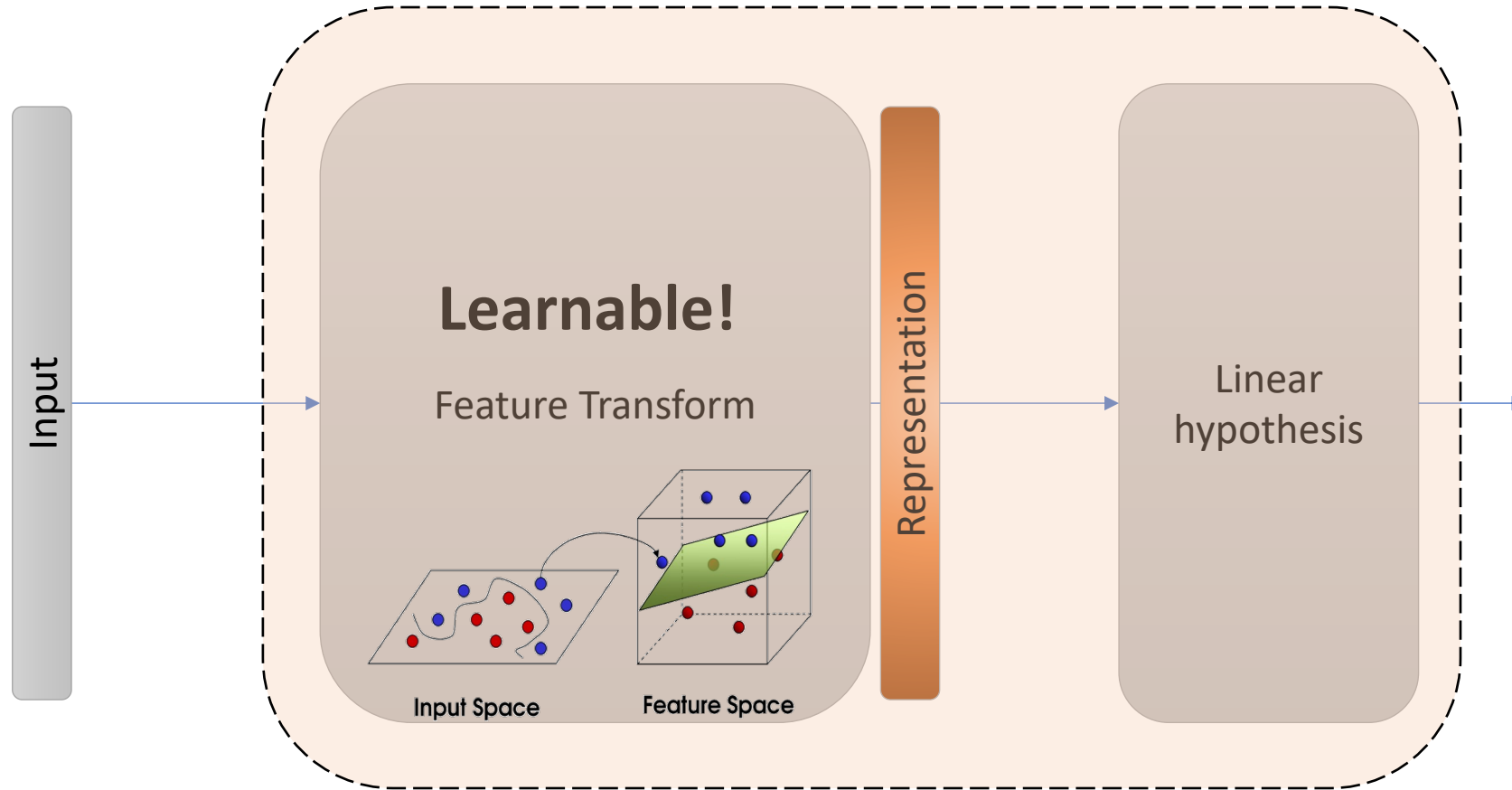
02:20) Why are neural networks better than others?

02:35) Demo

02:45) ---- Break ----

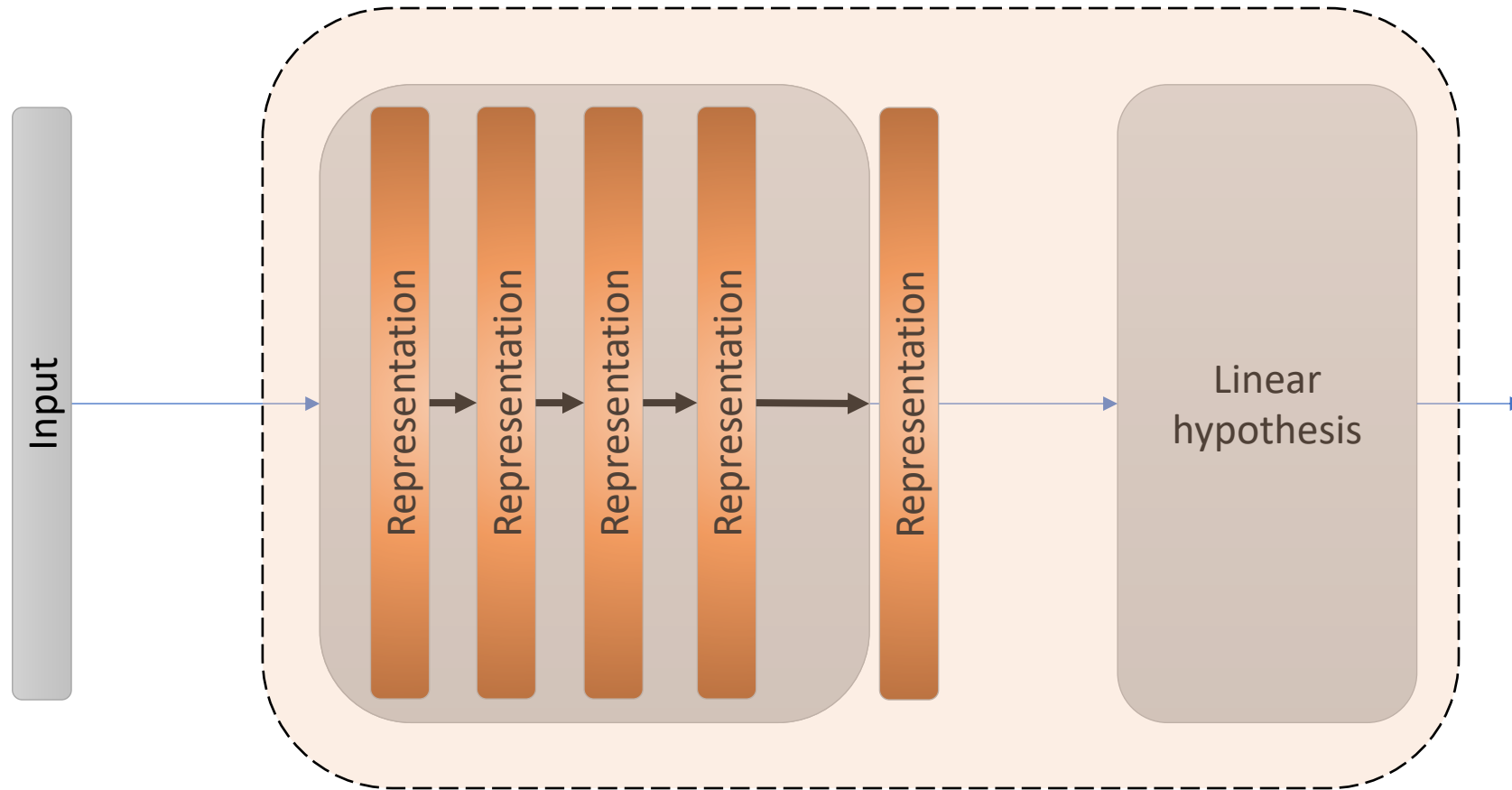
03:00) Tutorial

Feature transform



Non-linear hypothesis!

Feature transform



Non-linear hypothesis!

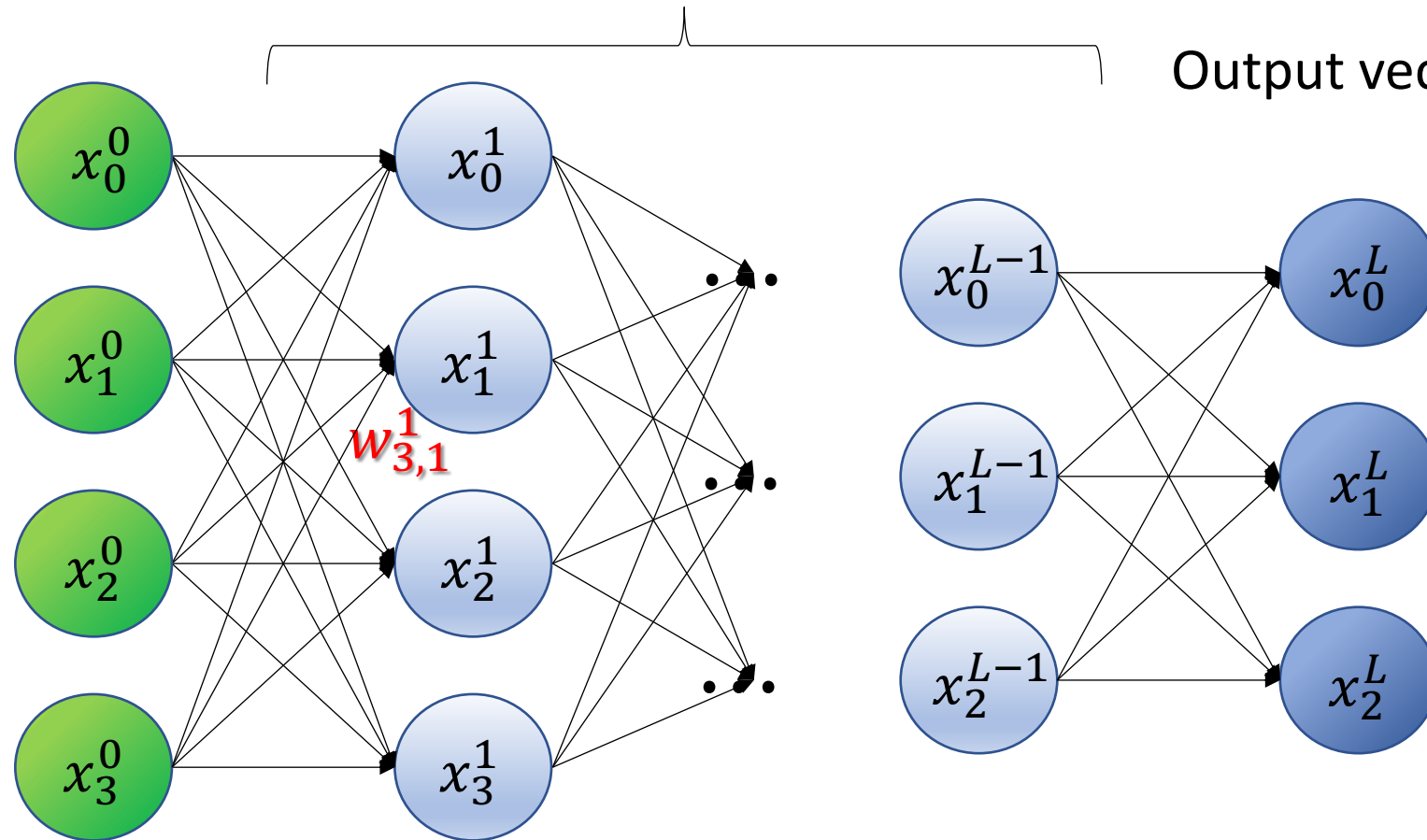
Artificial Neural Networks

Vaguely inspired by biological neural networks

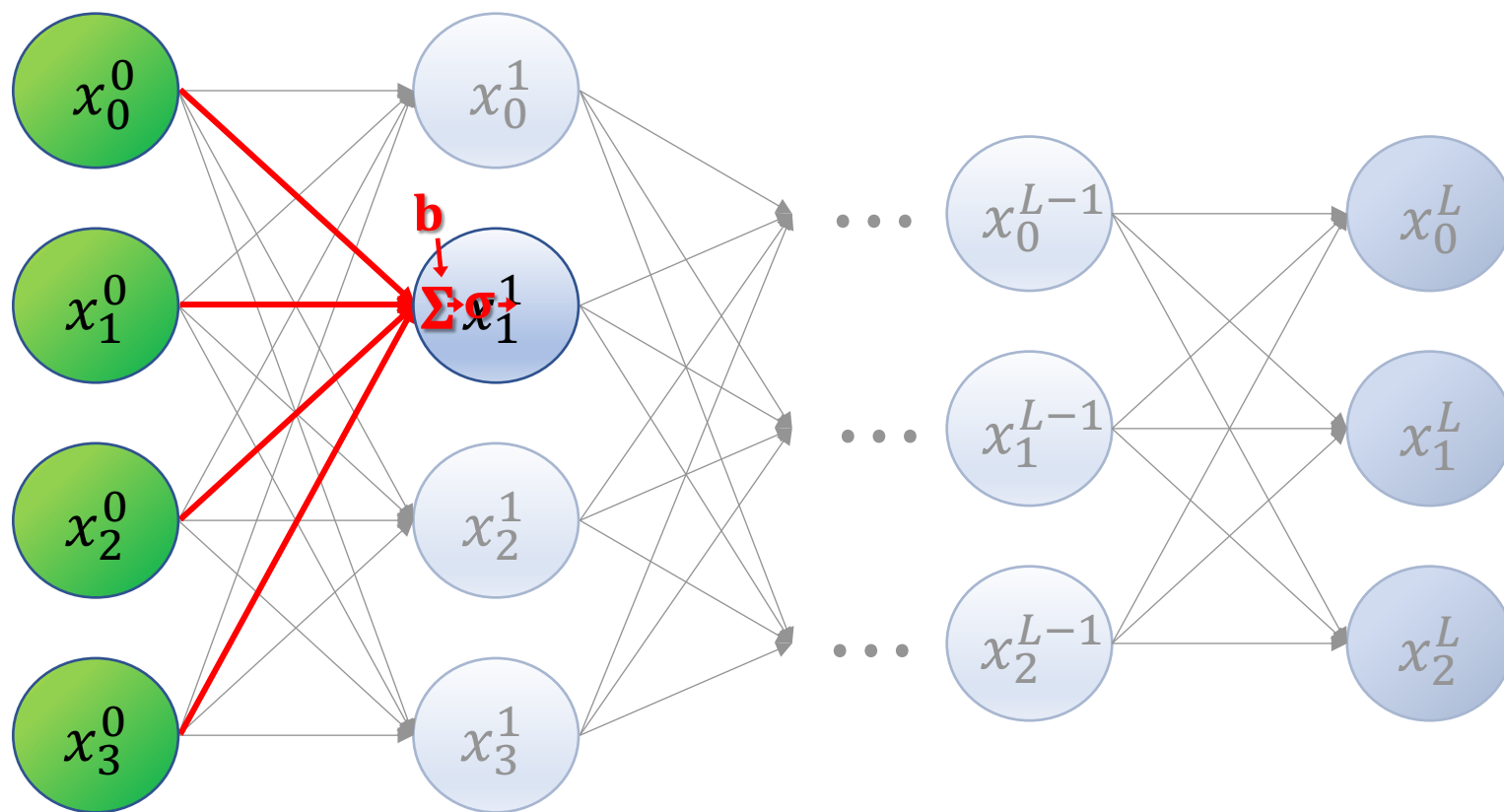
Input vector

Hidden layers

Output vector

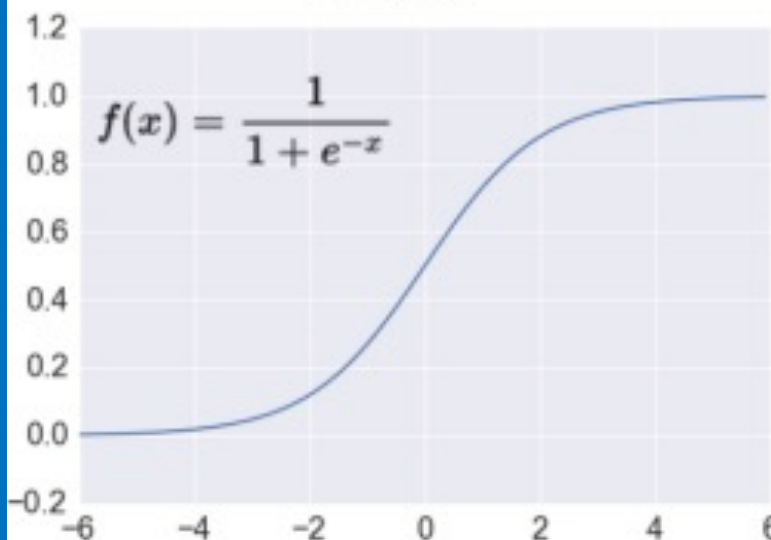


$$x_j^l = \sigma \left(\sum_i w_{ij}^l \cdot x_i^{l-1} + b_j^l \right) \quad \mathbf{x}^l = \boldsymbol{\sigma} \left(\mathbf{W}^{lT} \mathbf{x}^{l-1} + \mathbf{b}^l \right)$$

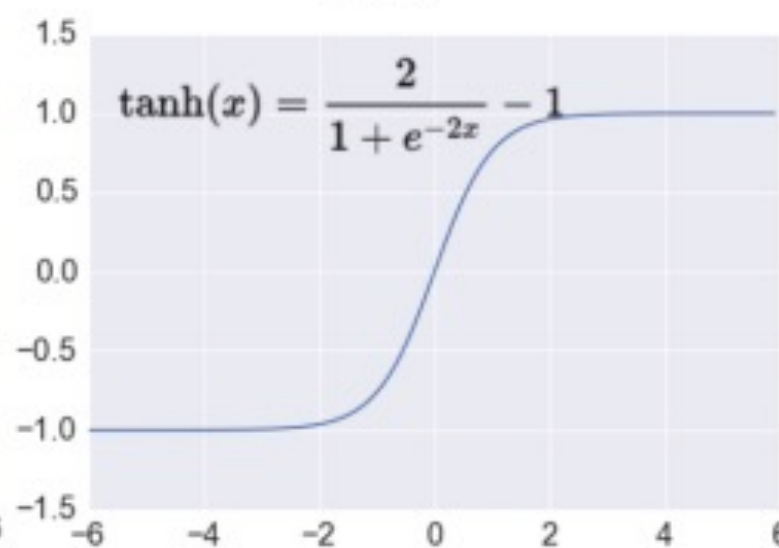


$$x_j^l = \sigma \left(\sum_i w_{ij}^l \cdot x_i^{l-1} + b_j^l \right) \quad \mathbf{x}^l = \boldsymbol{\sigma} \left(\mathbf{W}^{lT} \mathbf{x}^{l-1} + \mathbf{b}^l \right)$$

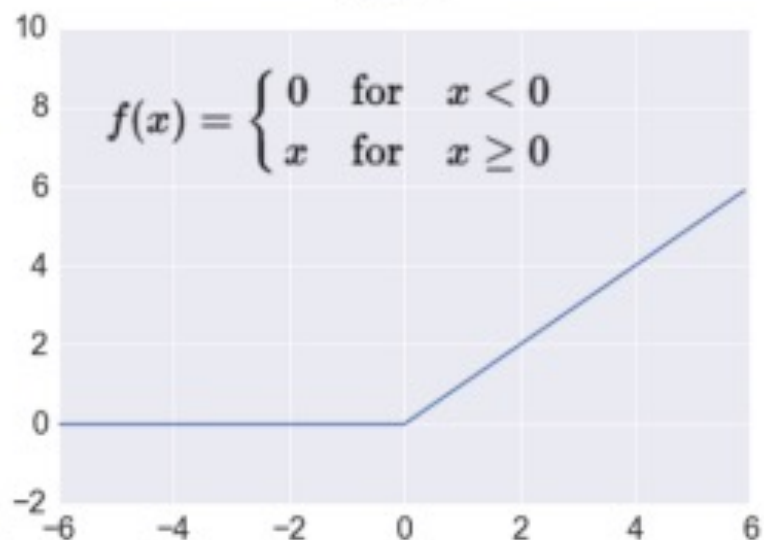
Sigmoid



TanH



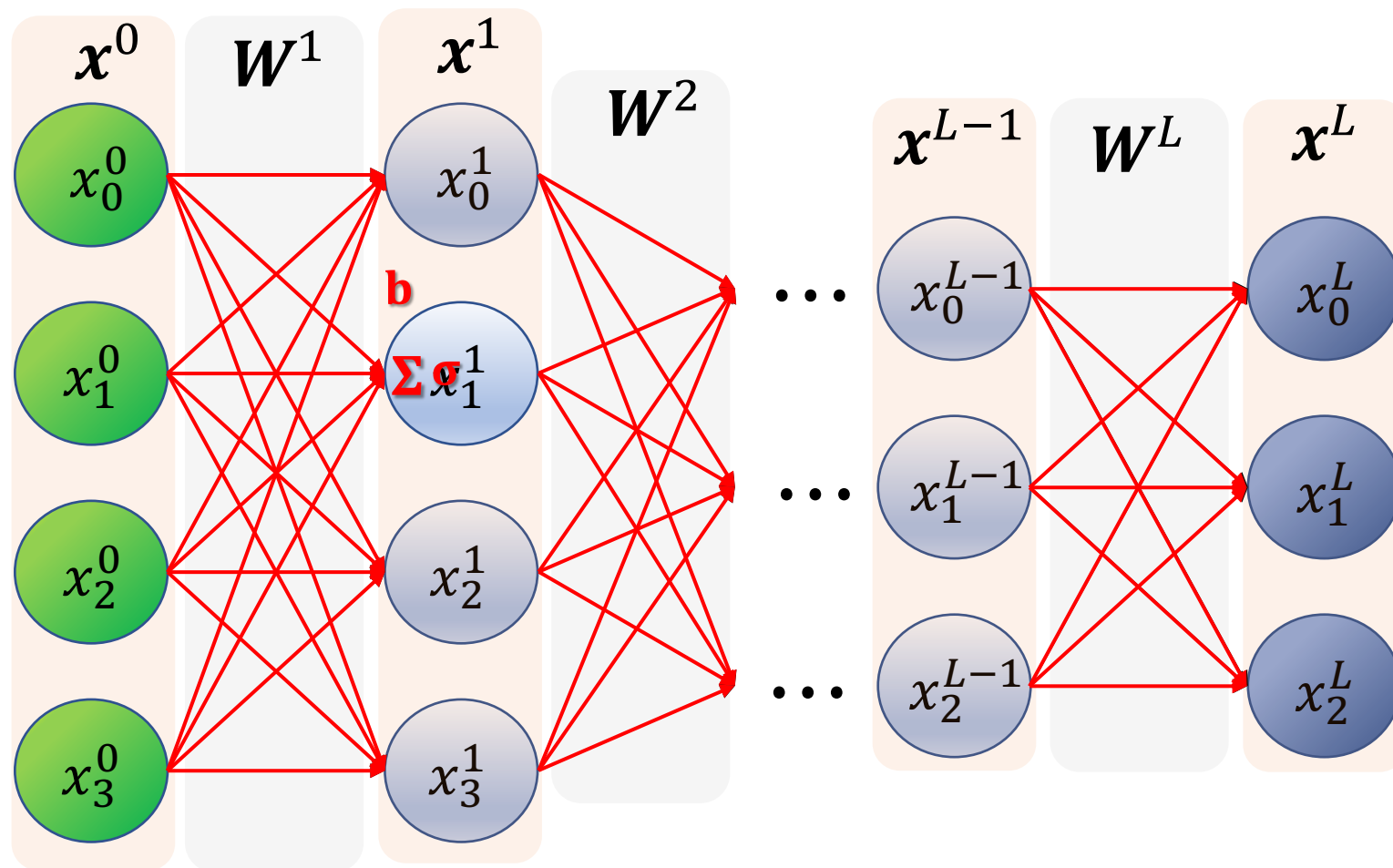
ReLU



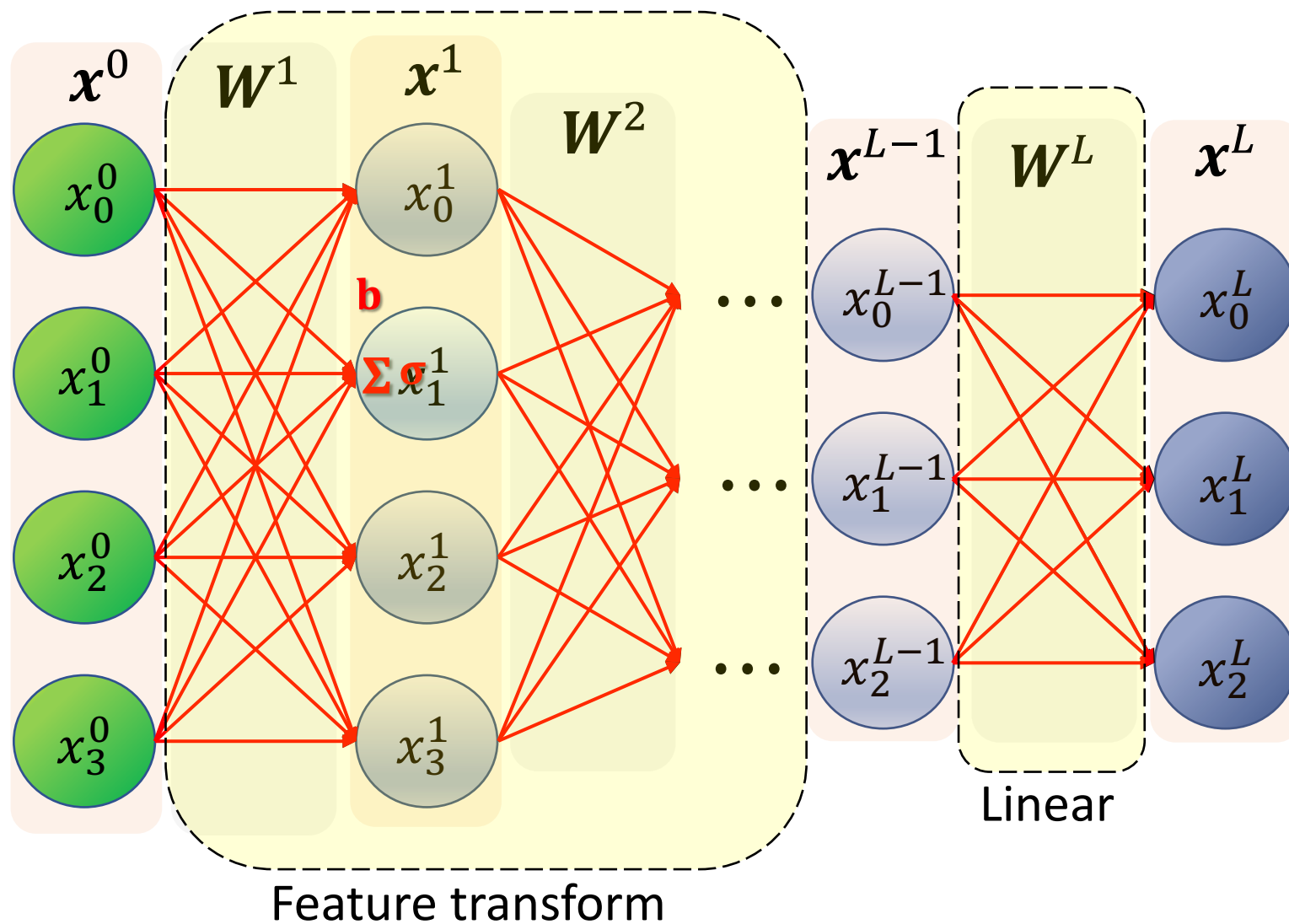
λ_3

λ_3

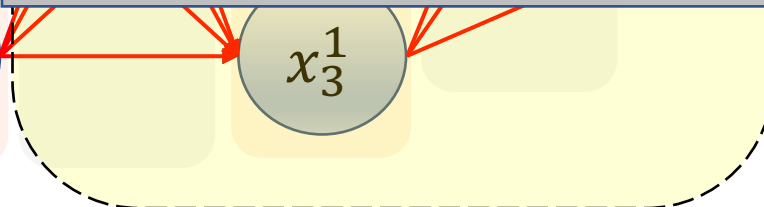
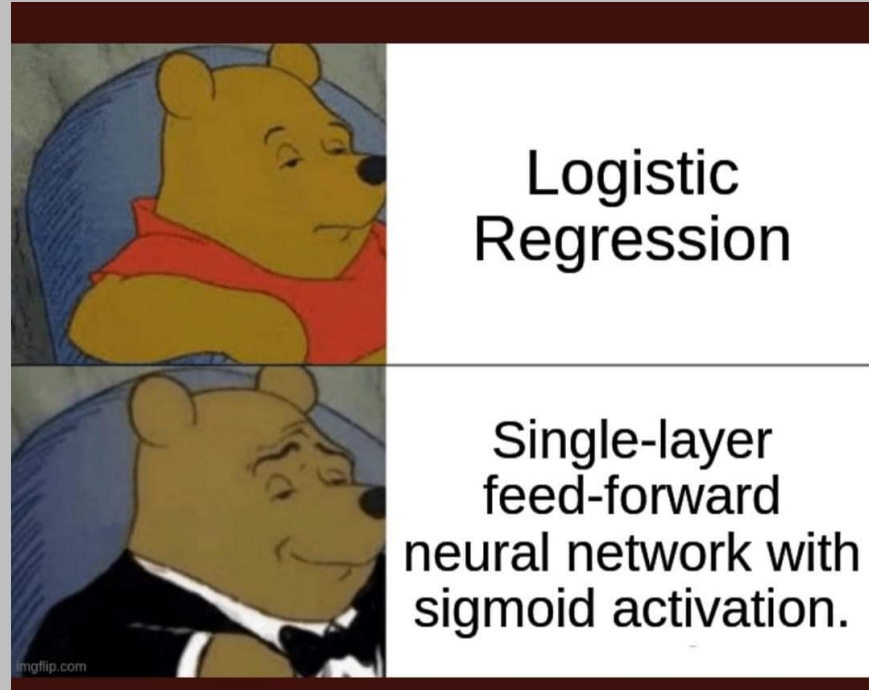
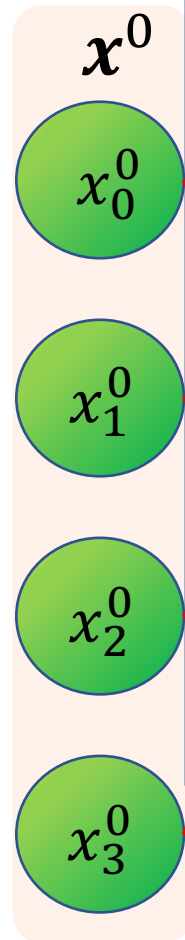
$$x_j^l = \sigma \left(\sum_i w_{ij}^l \cdot x_i^{l-1} + b_j^l \right) \qquad \mathbf{x}^l = \boldsymbol{\sigma} \left(\mathbf{W}^{lT} \mathbf{x}^{l-1} + \mathbf{b}^l \right)$$



$$x_j^l = \sigma \left(\sum_i w_{ij}^l \cdot x_i^{l-1} + b_j^l \right) \quad \mathbf{x}^l = \boldsymbol{\sigma} \left(\mathbf{W}^{lT} \mathbf{x}^{l-1} + \mathbf{b}^l \right)$$



$$x_j^l = \sigma \left(\text{Q: What do you call a single layered net? } b^l \right)$$

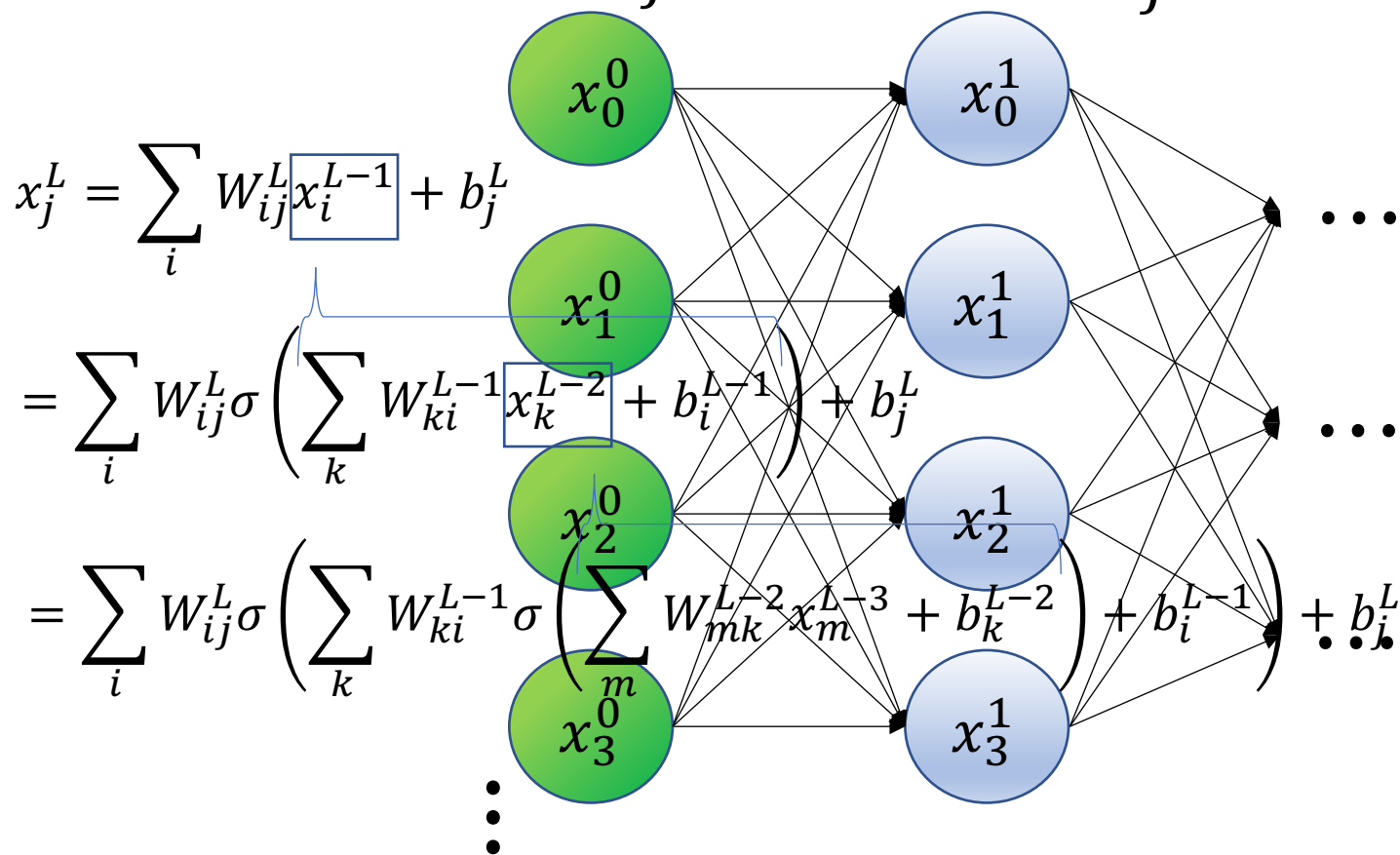


Feature transform

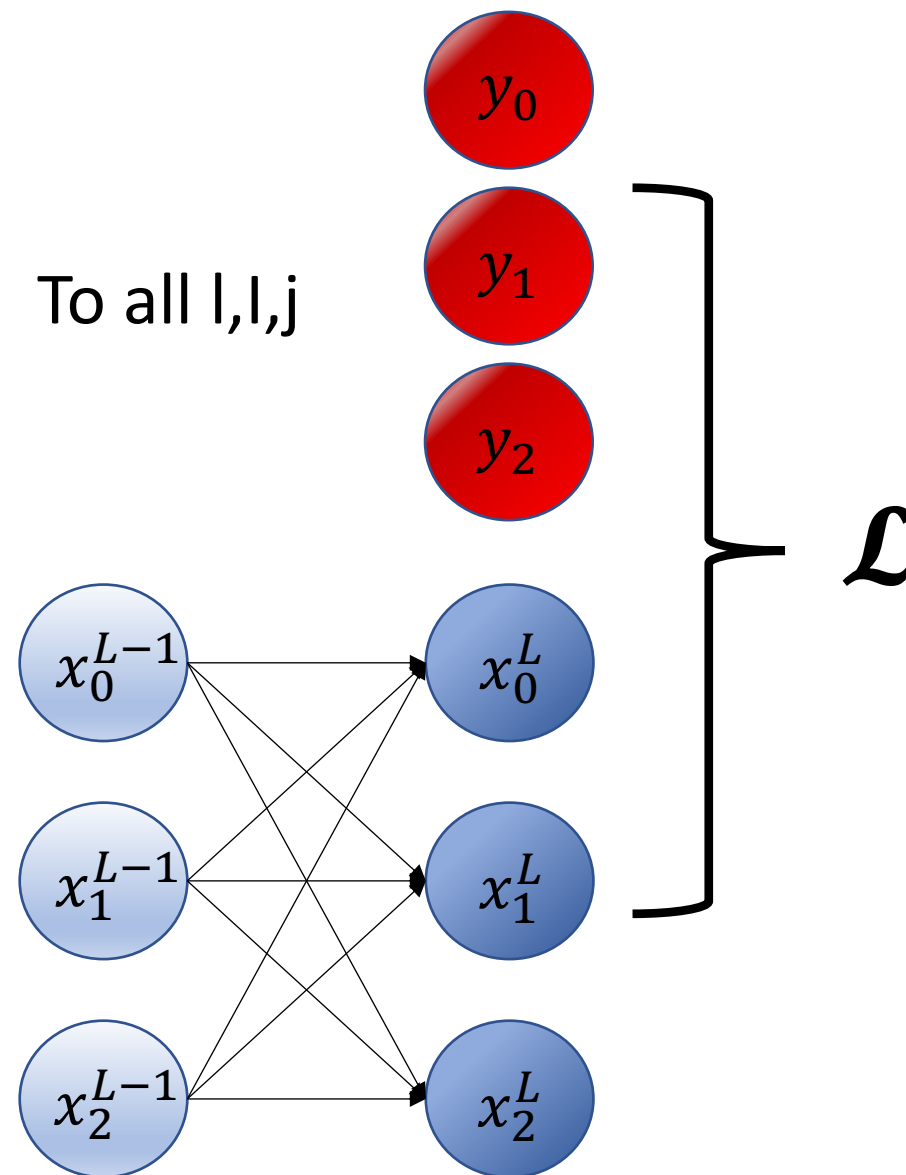
Linear

Learning by SGD

We need $\frac{\partial \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}, \mathbf{y})}{\partial w_{ij}^l}$, $\frac{\partial \mathcal{L}(\boldsymbol{\theta}; \mathbf{x}, \mathbf{y})}{\partial b_j^l}$



To all l, i, j



Chain rule reminder

$$f(g(x))' = f'(g(x)) \cdot g'(x)$$

$$y = f(x), \quad z = g(y)$$

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

$$\frac{dy}{dx} \cdot \frac{dz}{dy} = \frac{dz}{dx}$$

$$4x + 2e^{2x} = \frac{dy_1}{dx} \cdot \frac{\partial z}{\partial y_1} = 3$$

$$\frac{dz(y_1, y_2)}{dx} = \frac{\partial z}{\partial y_1} \frac{dy_1}{dx} + \frac{\partial z}{\partial y_2} \frac{dy_2}{dx}$$

$3 \quad 4x + 2e^{2x} \quad 2e^x \quad e^x$

$= 6x^2 + 4e^{2x}$

$$e^x = \frac{dy_2}{dx} \cdot \frac{\partial z}{\partial y_2} = \frac{dz}{dx}$$

$2y_2 = 2e^x$

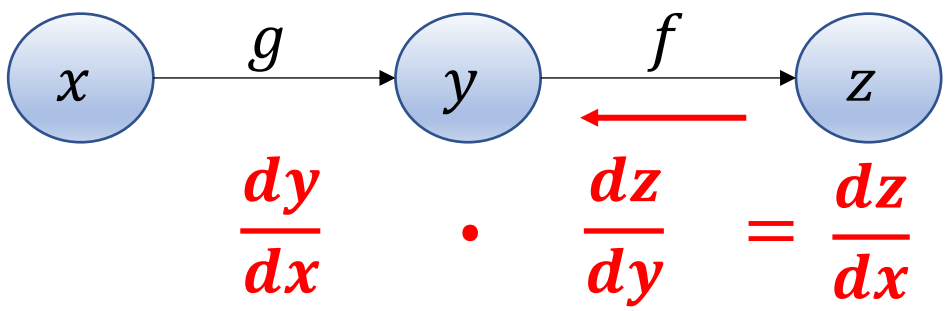
$\frac{dz(y_1, y_2)}{dx} = 12x + 8e^{2x}$

Chain rule reminder

$$f(g(x))' = f'(g(x)) \cdot g'(x)$$

$$y = f(x), \quad z = g(y)$$

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$



$$4x + 2e^{2x} = \frac{dy_1}{dx} \quad \cdot \quad \frac{\partial z}{\partial y_1} = 3$$

$$\frac{dz(y_1, y_2)}{dx} = \frac{\partial z}{\partial y_1} \frac{dy_1}{dx} + \frac{\partial z}{\partial y_2} \frac{dy_2}{dx}$$

3
 $4x + 2e^{2x}$
 $2e^x$
 e^x

$= 6x^2 + 4e^{2x}$

$$+ \quad e^x = \frac{dy_2}{dx} \quad \cdot \quad \frac{\partial z}{\partial y_2} = 2y_2 = 2e^x = \frac{dz}{dx}$$

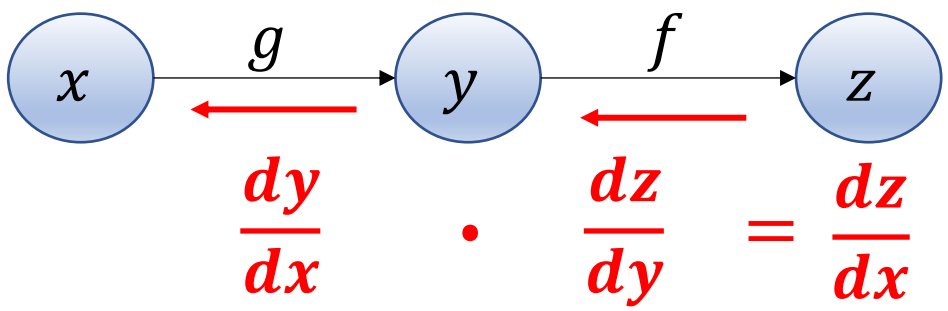
$\frac{dz(y_1, y_2)}{dx} = 12x + 8e^{2x}$

Chain rule reminder

$$f(g(x))' = f'(g(x)) \cdot g'(x)$$

$$y = f(x), \quad z = g(y)$$

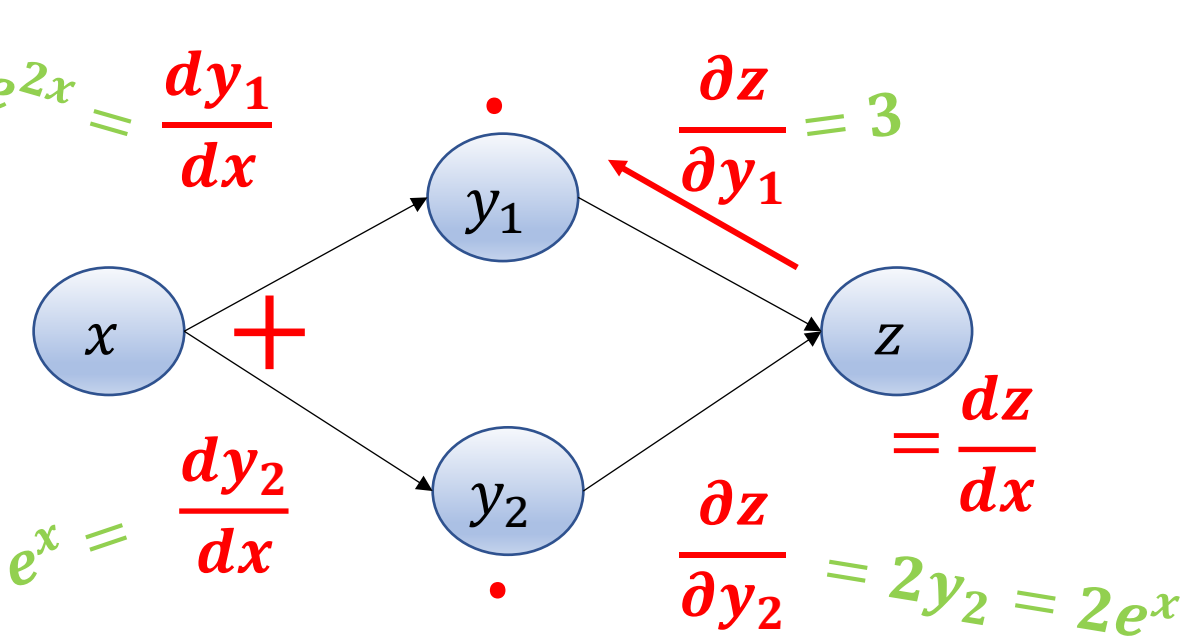
$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$



$$\frac{dz(y_1, y_2)}{dx} = \frac{\partial z}{\partial y_1} \frac{dy_1}{dx} + \frac{\partial z}{\partial y_2} \frac{dy_2}{dx}$$

Annotations for the above equation:
- Under $\frac{\partial z}{\partial y_1}$: 3
- Under $\frac{dy_1}{dx}$: $4x + 2e^{2x}$
- Under $\frac{\partial z}{\partial y_2}$: $2e^x$
- Under $\frac{dy_2}{dx}$: e^x

$= 6x^2 + 4e^{2x}$



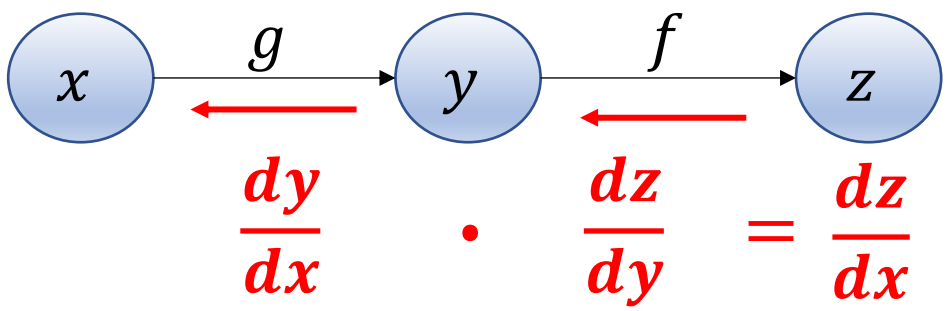
$\frac{dz(y_1, y_2)}{dx} = 12x + 8e^{2x}$

Chain rule reminder

$$f(g(x))' = f'(g(x)) \cdot g'(x)$$

$$y = f(x), \quad z = g(y)$$

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$



$$y_1 = 2x^2 + e^{2x}$$

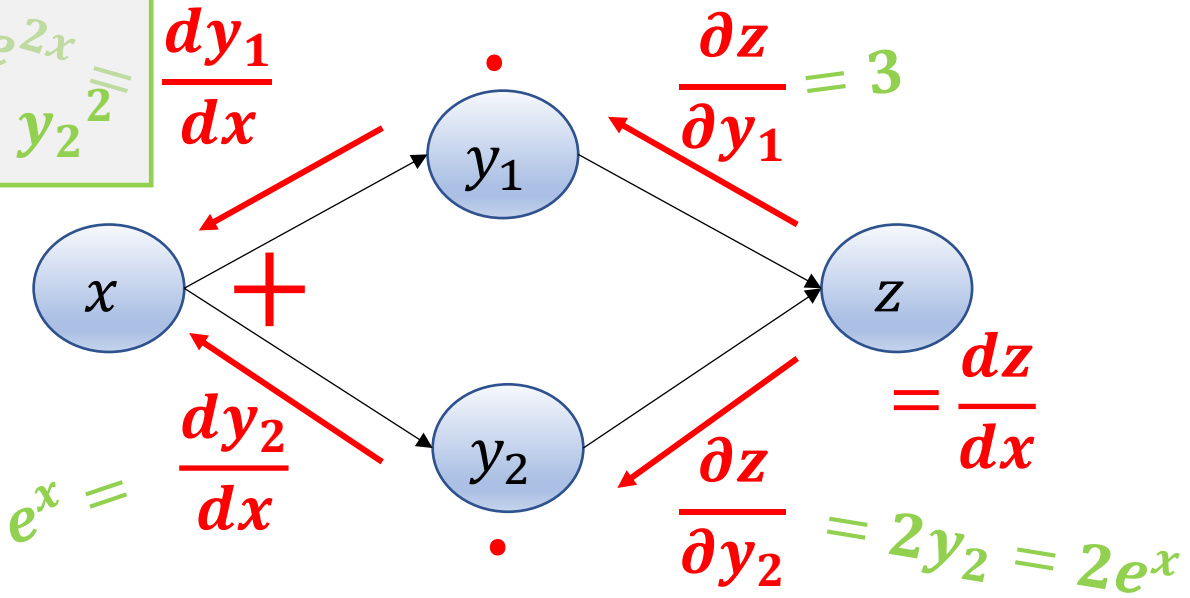
$$y_2 = 4xe^x + 2e^{2x}$$

$$z = 3y_1 + y_2^2$$

$$\frac{dz(y_1, y_2)}{dx} = \frac{\partial z}{\partial y_1} \frac{dy_1}{dx} + \frac{\partial z}{\partial y_2} \frac{dy_2}{dx}$$

$3 \qquad 4x + 2e^{2x} \qquad 2e^x \qquad e^x$

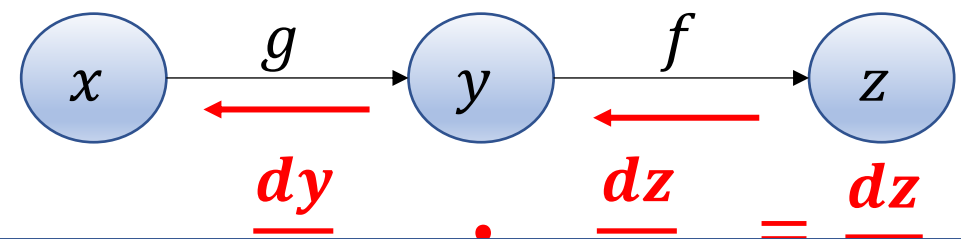
$= 6x^2 + 4e^{2x}$



$\frac{dz(y_1, y_2)}{dx} = 12x + 8e^{2x}$

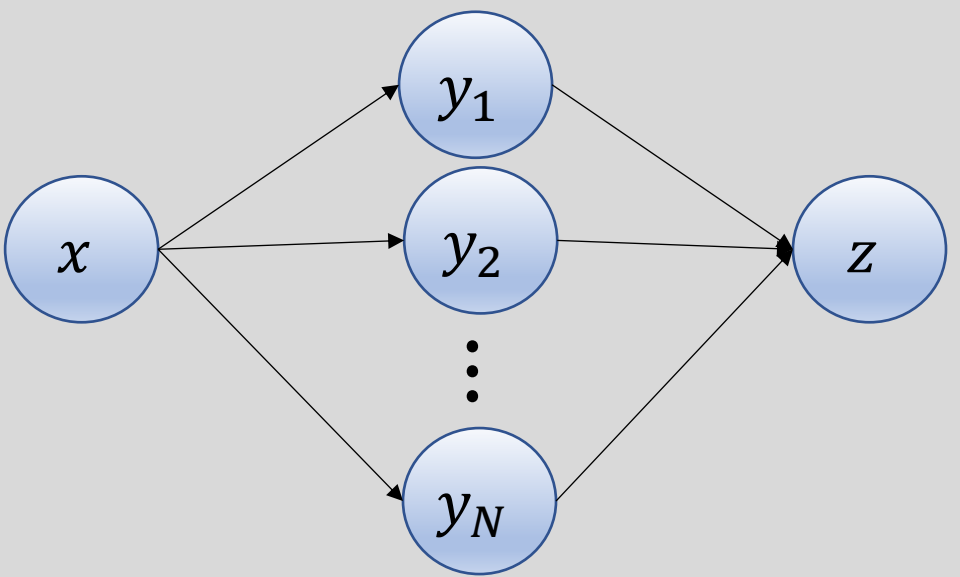
Chain rule reminder

$$f(g(x))' = f'(g(x)) \cdot g'(x)$$



Conclusion:

$$\frac{df(y_1, y_2 \dots y_N)}{dx} = \sum_n \frac{\partial f}{\partial y_n} \frac{dy_n}{dx}$$



$$= 6x^2 + 4e^{2x}$$

$$\frac{dz(y_1, y_2)}{dx} = 12x + 8e^{2x}$$

Back Propagation - preliminaries

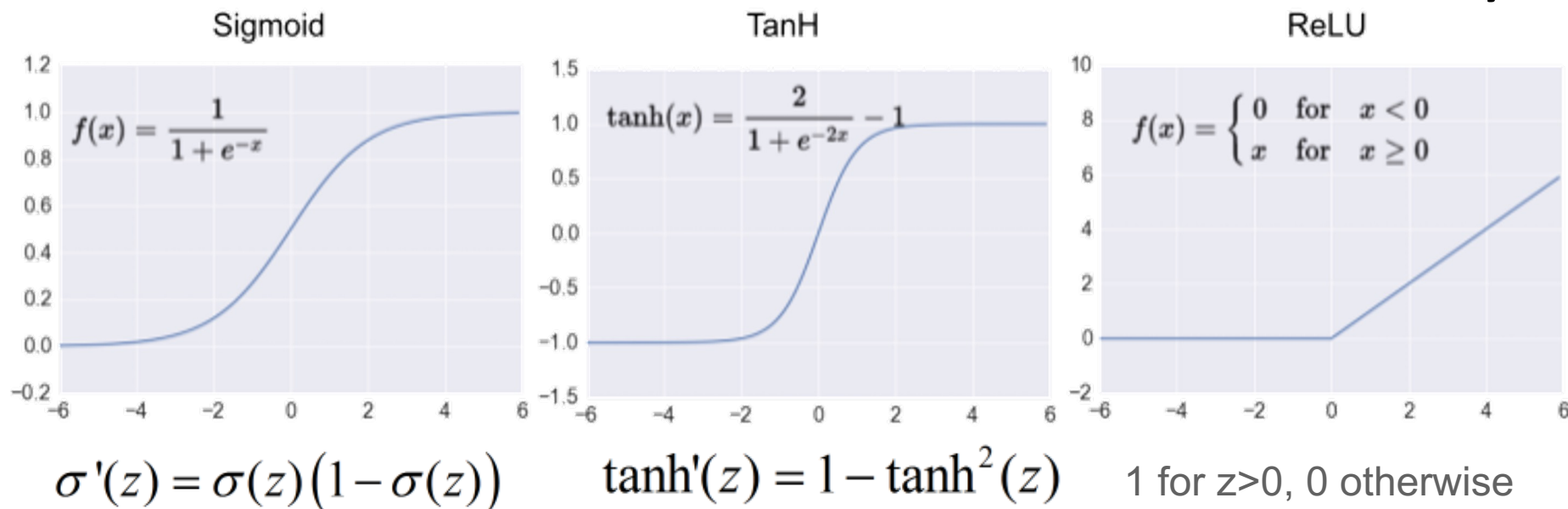
$$x_j^l = \sigma \left(\underbrace{\sum_i w_{ij}^l \cdot x_i^{l-1}}_{z_j^l} + b_j \right)$$

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^l} = \boxed{\frac{\partial \mathcal{L}}{\partial x_j^l}} \cdot \boxed{\frac{\partial x_j^l}{\partial w_{ij}^l}}$$

$\triangleq g_j^l$
Obtained by
backprop

Easy!
 $x_i^{l-1} \cdot \sigma'(z_j^l)$

Derivatives of common activations are easy!



Back Propagation

$$g_j^l \triangleq \frac{\partial \mathcal{L}}{\partial x_j^l}$$

$$= \sum_k \left[\frac{\partial \mathcal{L}}{\partial x_k^{l+1}} \right] \cdot \left[\frac{\partial x_k^{l+1}}{\partial x_j^l} \right]$$

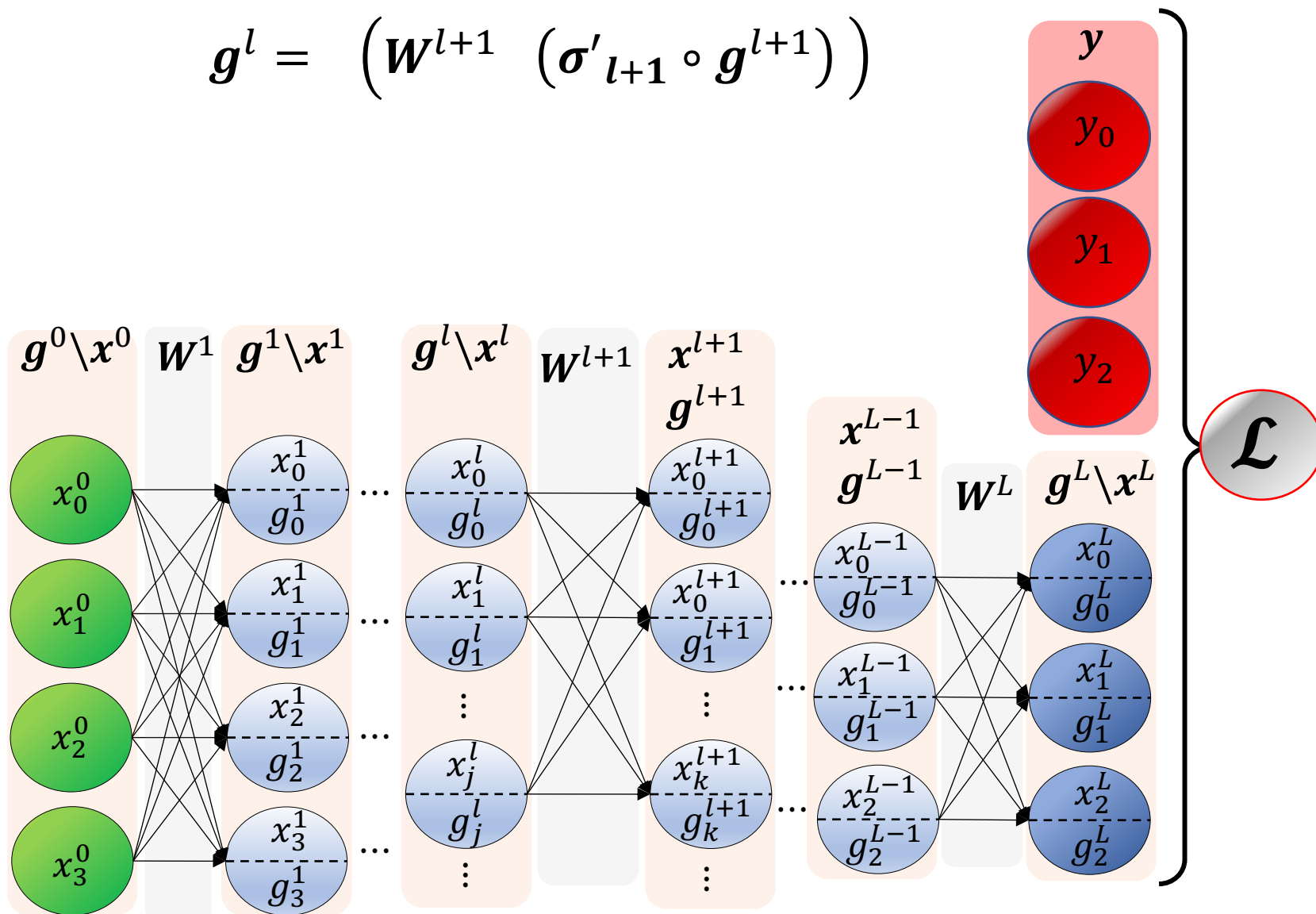
$$= \sum_k \underbrace{g_k^{l+1}}_{g_k^{l+1}} \cdot \underbrace{w_{jk}^{l+1} \cdot \sigma'(z_k^{l+1})}_{w_{jk}^{l+1} \cdot \sigma'(z_k^{l+1})}$$

$$\mathbf{x}^l = \sigma \left(\mathbf{W}^{lT} \mathbf{x}^{l-1} + \mathbf{b} \right)$$

$$\mathbf{g}^l = \left(\mathbf{W}^{l+1} \left(\sigma'_{l+1} \circ \mathbf{g}^{l+1} \right) \right)$$

Stopping criterion:

$$g_j^L = \frac{\partial \mathcal{L}}{\partial x_j^L}$$



Back Propagation

$$\begin{aligned} g_j^l &\triangleq \frac{\partial \mathcal{L}}{\partial x_j^l} \\ &= \sum_k \boxed{\frac{\partial \mathcal{L}}{\partial x_k^{l+1}}} \cdot \boxed{\frac{\partial x_k^{l+1}}{\partial x_j^l}} \\ &= \sum_k \underbrace{g_k^{l+1}} \cdot \underbrace{w_{jk}^{l+1} \cdot \sigma'(z_k^{l+1})} \end{aligned}$$

Stopping criterion:

$$g_j^L = \frac{\partial \mathcal{L}}{\partial x_j^L}$$

$$x^l = \sigma \left(W^{lT} x^{l-1} + b \right)$$

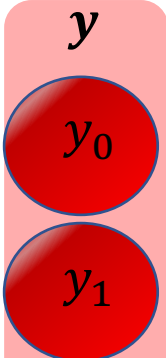
$$g^l = \left(W^{l+1} \left(\sigma'_{l+1} \circ g^{l+1} \right) \right)$$

Linear
activation

Not
Transposed

Multiply by
activation
derivative

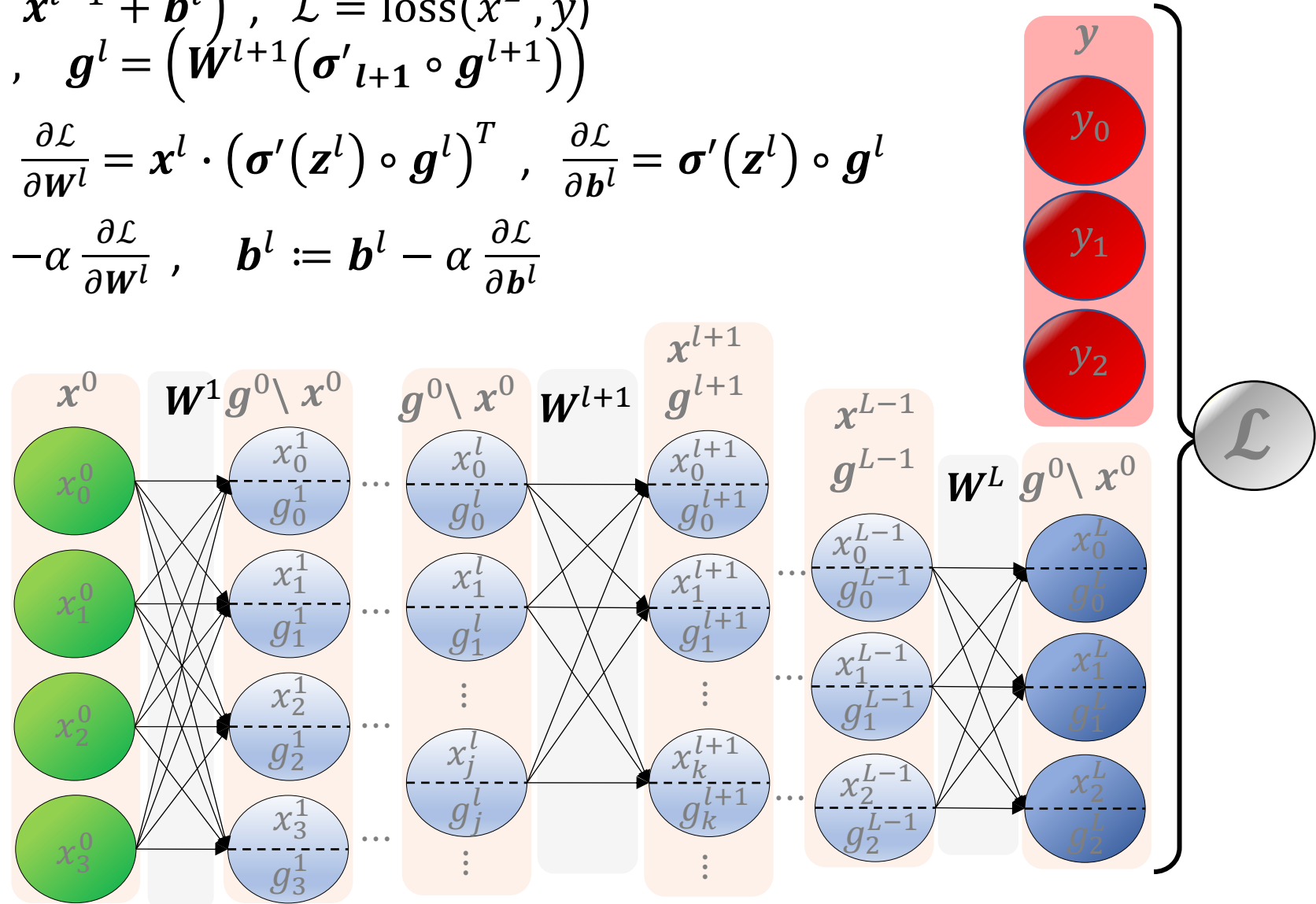
No bias



$\mathcal{L}$

- Initialize weights
- Repeat until convergence:
 1. Sample a batch from the data: $\{(\mathbf{x}_i, \mathbf{y}_i) \dots\}$
 2. Forward pass: $\mathbf{x}^l = \sigma(\mathbf{W}^{lT} \mathbf{x}^{l-1} + \mathbf{b}^l)$, $\mathcal{L} = \text{loss}(\mathbf{x}^L, \mathbf{y})$
 3. Backward pass: $\mathbf{g}^L = \frac{\partial \mathcal{L}}{\partial \mathbf{x}^L}$, $\mathbf{g}^l = \left(\mathbf{W}^{l+1} (\sigma'_{l+1} \circ \mathbf{g}^{l+1}) \right)$
 4. Calculate weights gradient: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^l} = \mathbf{x}^l \cdot (\sigma'(\mathbf{z}^l) \circ \mathbf{g}^l)^T$, $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^l} = \sigma'(\mathbf{z}^l) \circ \mathbf{g}^l$
 5. Update weights: $\mathbf{W}^l := \mathbf{W}^l - \alpha \frac{\partial \mathcal{L}}{\partial \mathbf{W}^l}$, $\mathbf{b}^l := \mathbf{b}^l - \alpha \frac{\partial \mathcal{L}}{\partial \mathbf{b}^l}$

Back Propagation



Let's get more generic



Yann LeCun

January 5, 2018 · 🌐

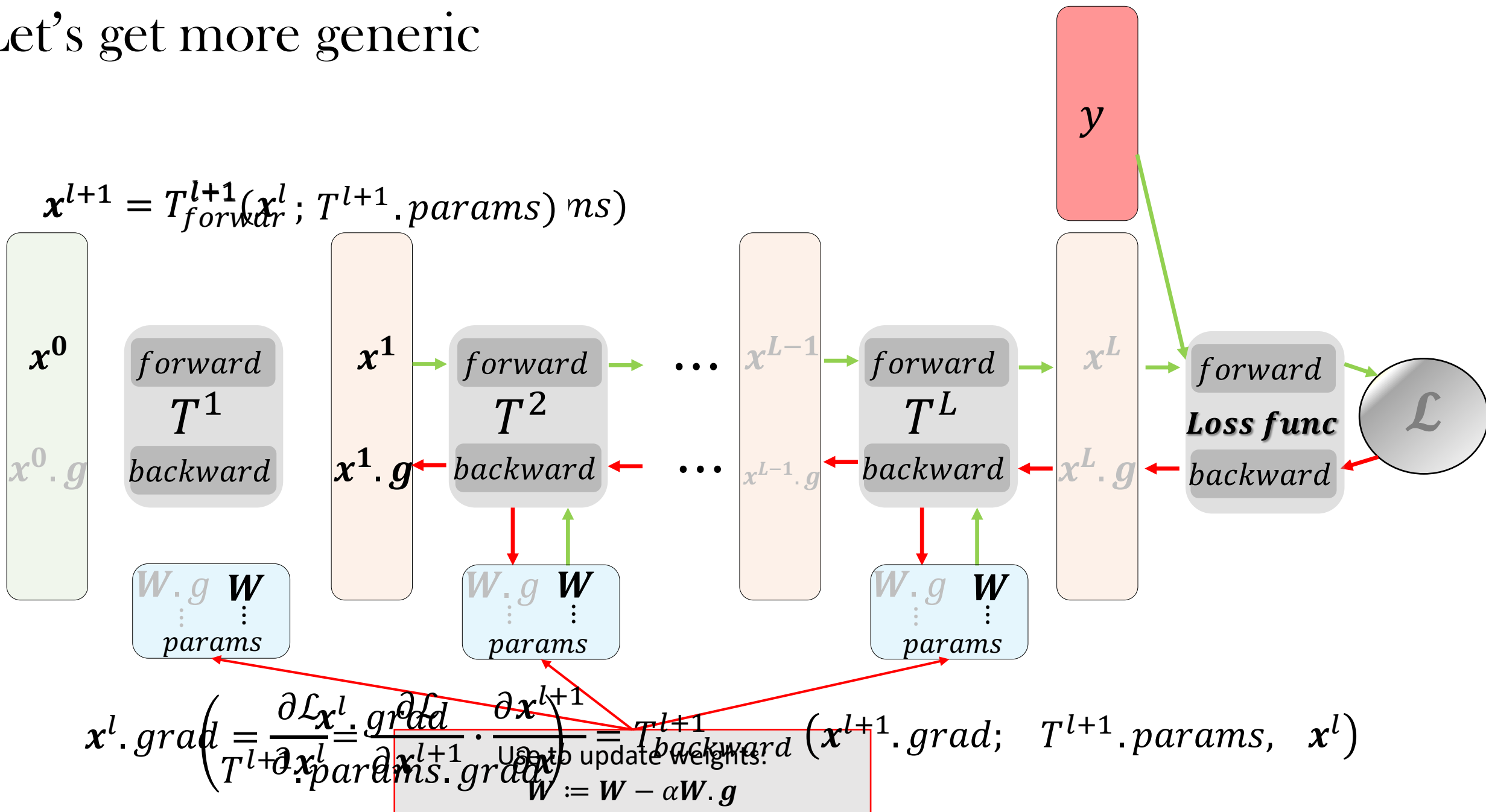


OK, Deep Learning has outlived its usefulness as a buzz-phrase.
Deep Learning est mort. Vive **Differentiable Programming!**

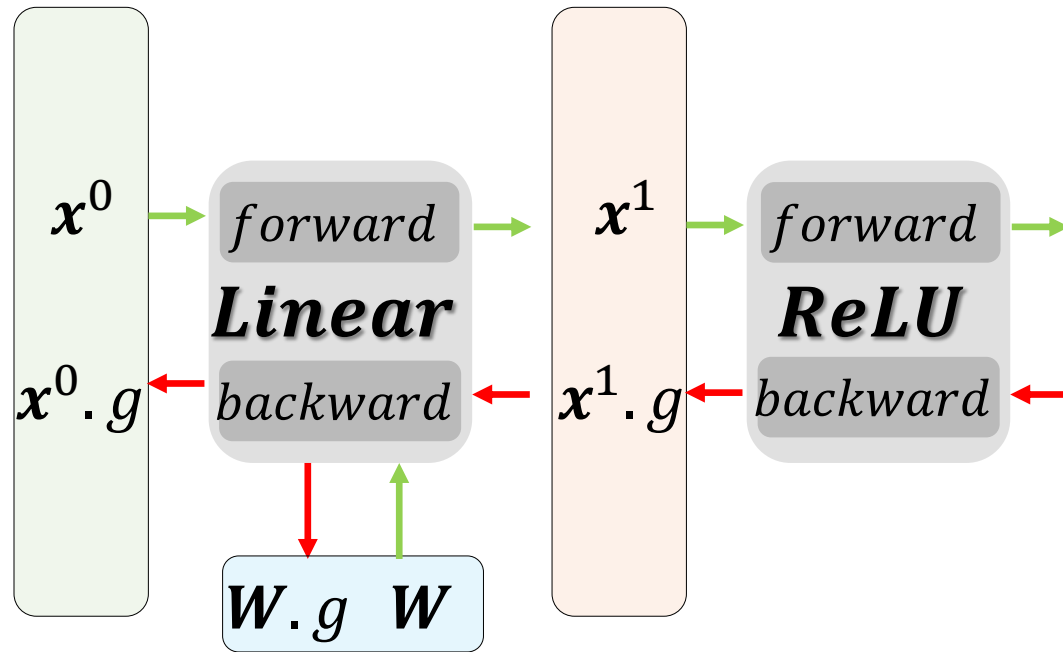
Yeah, Differentiable Programming is little more than a rebranding of the modern collection Deep Learning techniques, the same way Deep Learning was a rebranding of the modern incarnations of neural nets with more than two layers.

But the important point is that people are now building a new kind of software by assembling networks of **parameterized functional blocks** and by training them from examples using some form of gradient-based optimization.

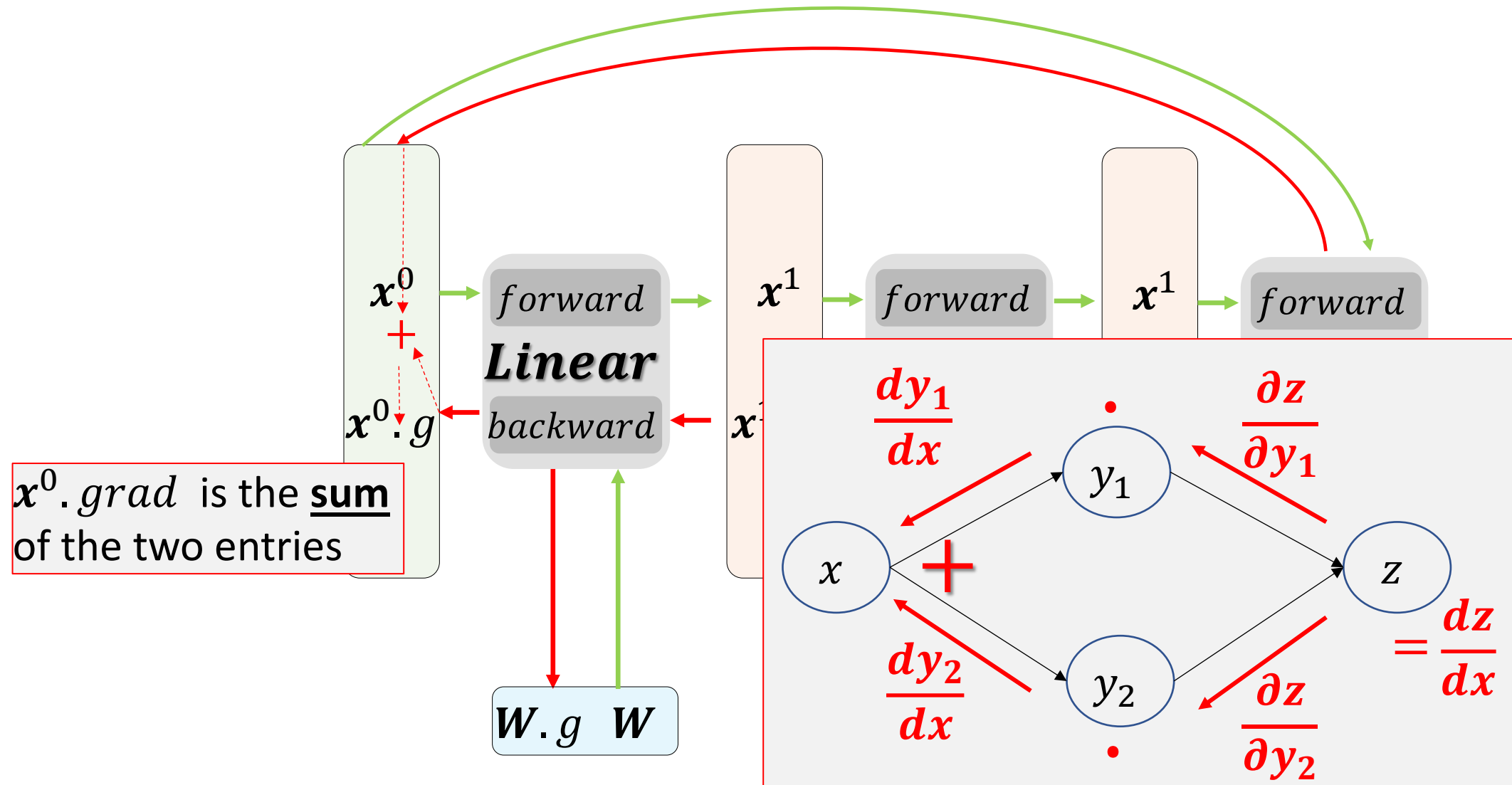
Let's get more generic



Example: Standard layer

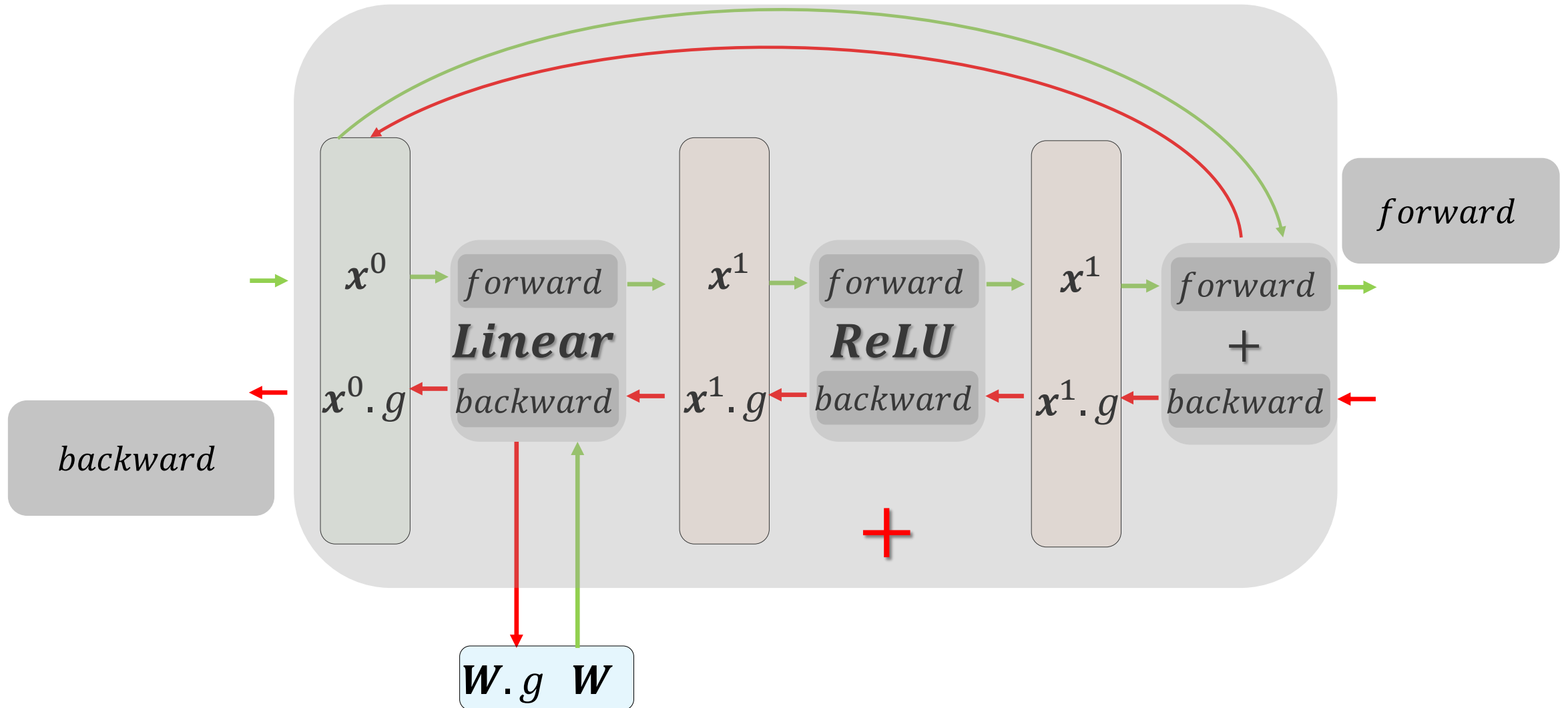


BTW : You can backprop any DAG!



BTW : You can backprop any DAG!

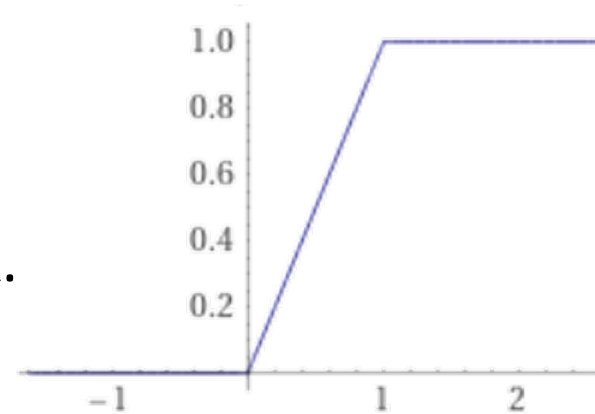
BTW2: Layers (NN modules) can be nested!



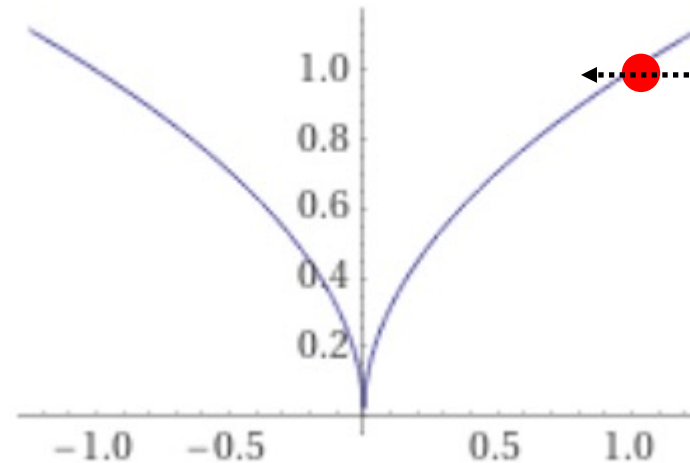
Q: Can we use any function inside a network?

- Has to be differentiable with respect to input and params
- Implications to optimization can be deadly:

- 0-1 clamp layer in the end of the network.
 - Ideas how to solve this?



- Square root loss ($L_{1/2}$ norm)



Q: Can we use any function inside a network?

**Be creative,
but always watch your back(prop)!**



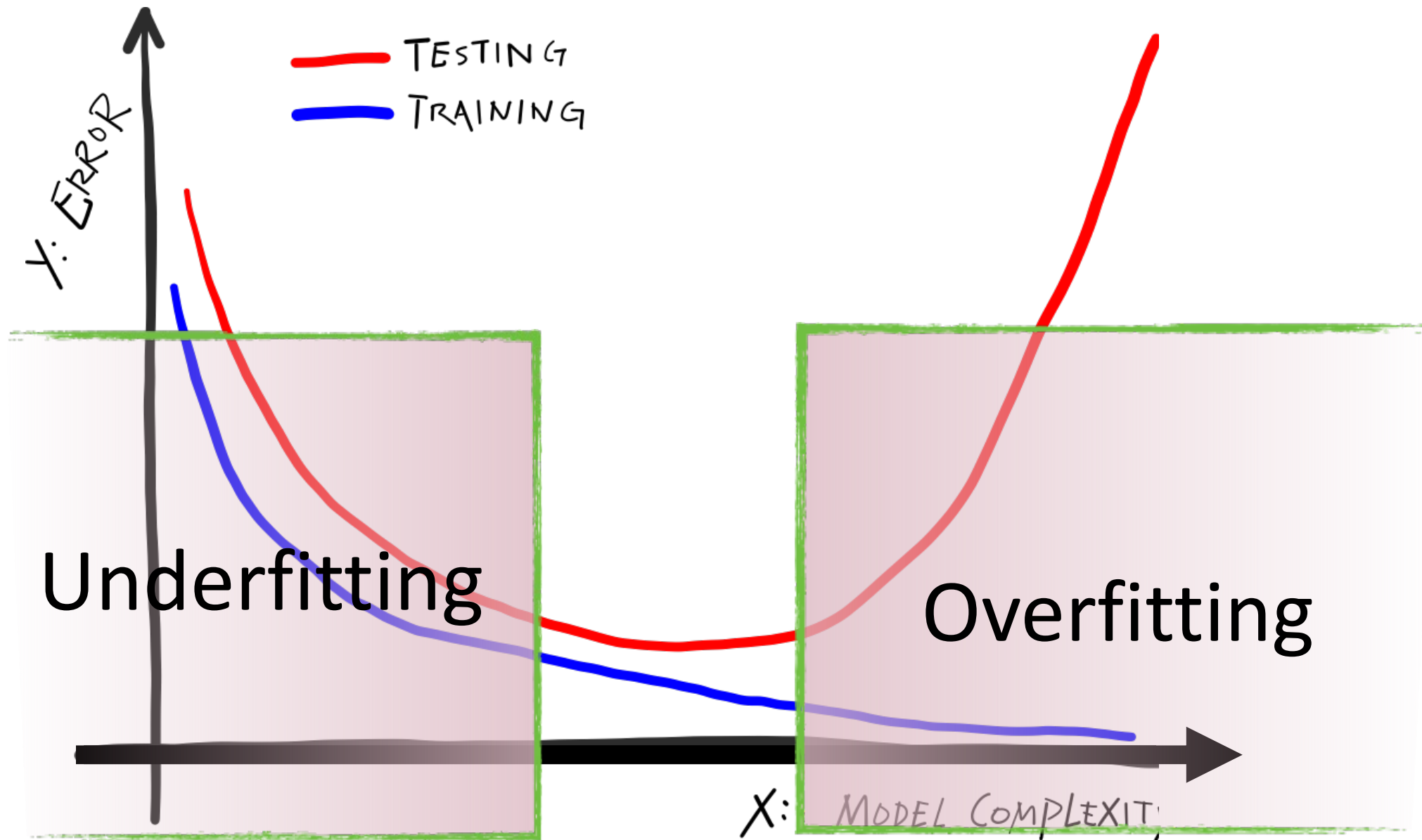
...

Now, let's talk about generalization

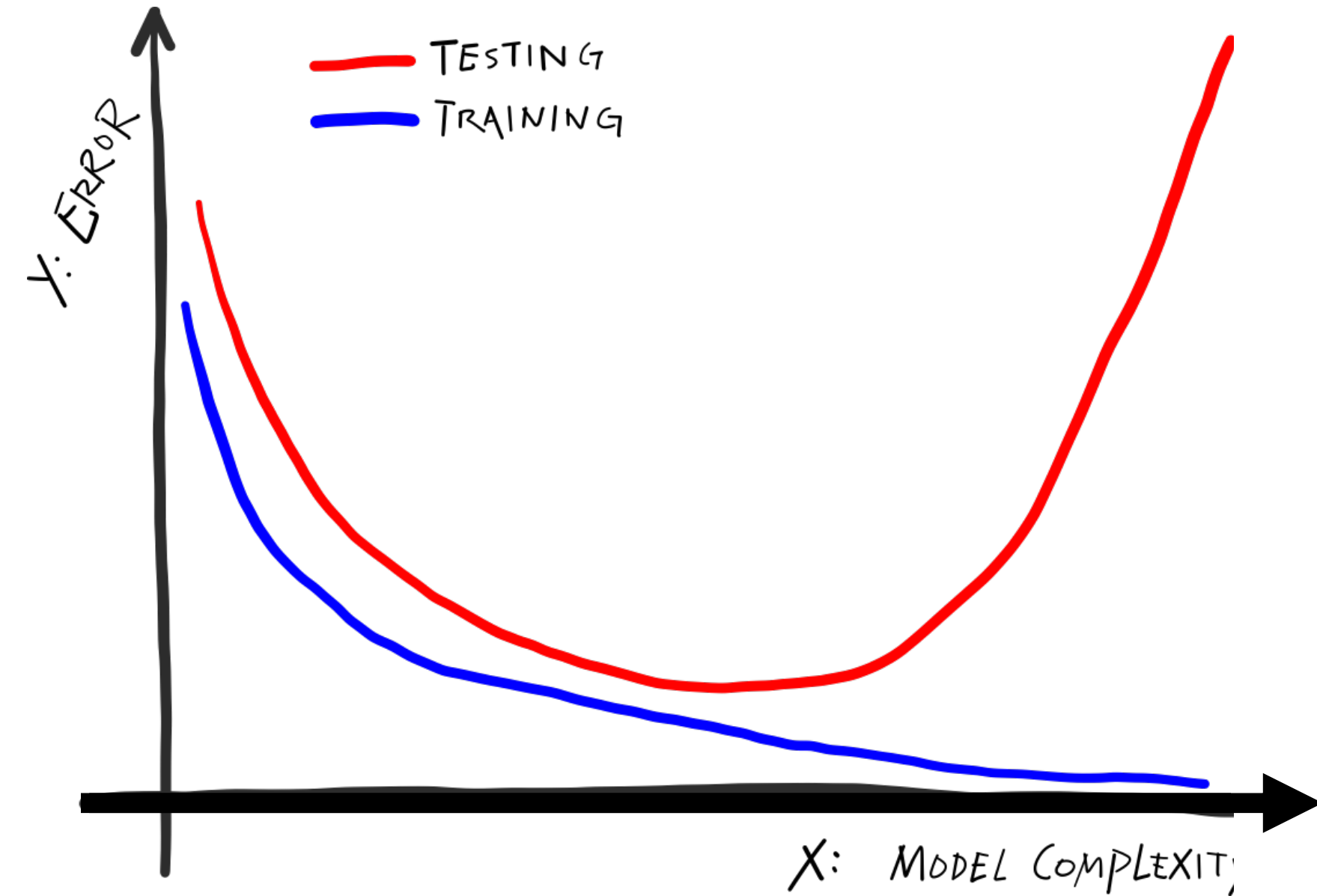
Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, Oriol Vinyals;
UNDERSTANDING DEEP LEARNING REQUIRES RETHINKING GENERALIZATION;
ICLR 2017

Some slides taken from the ICLR oral

Model-Complexity determines Generalization



Model-Complexity determines Generalization



Deep
Learning



Networks are bigger because they train on more data

Parameter Count

Num Training Samples

test error

50

37.5

25

12.5

0

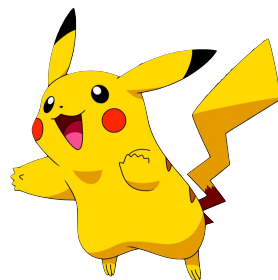
MLP 1x512

p/n: 24



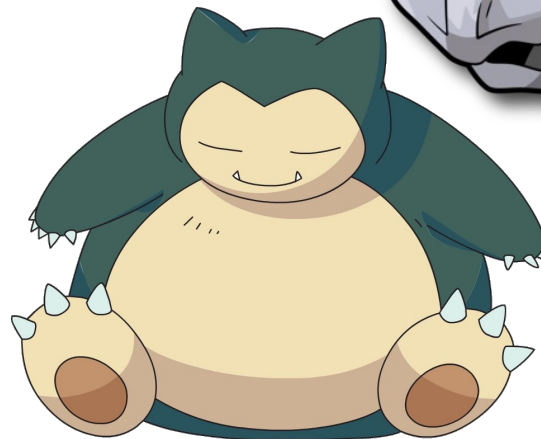
Alexnet

p/n: 28



Inception

p/n: 33



Wide Resnet

p/n: 179



MLP 1x512

Alexnet

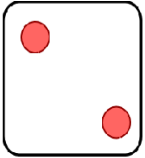
Inception

Wide Resnet

Randomization Test



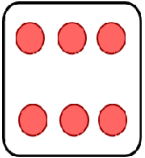
Dog



Cat



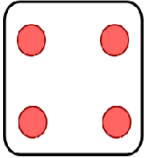
Flower



Dog



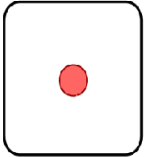
Cat



Bus



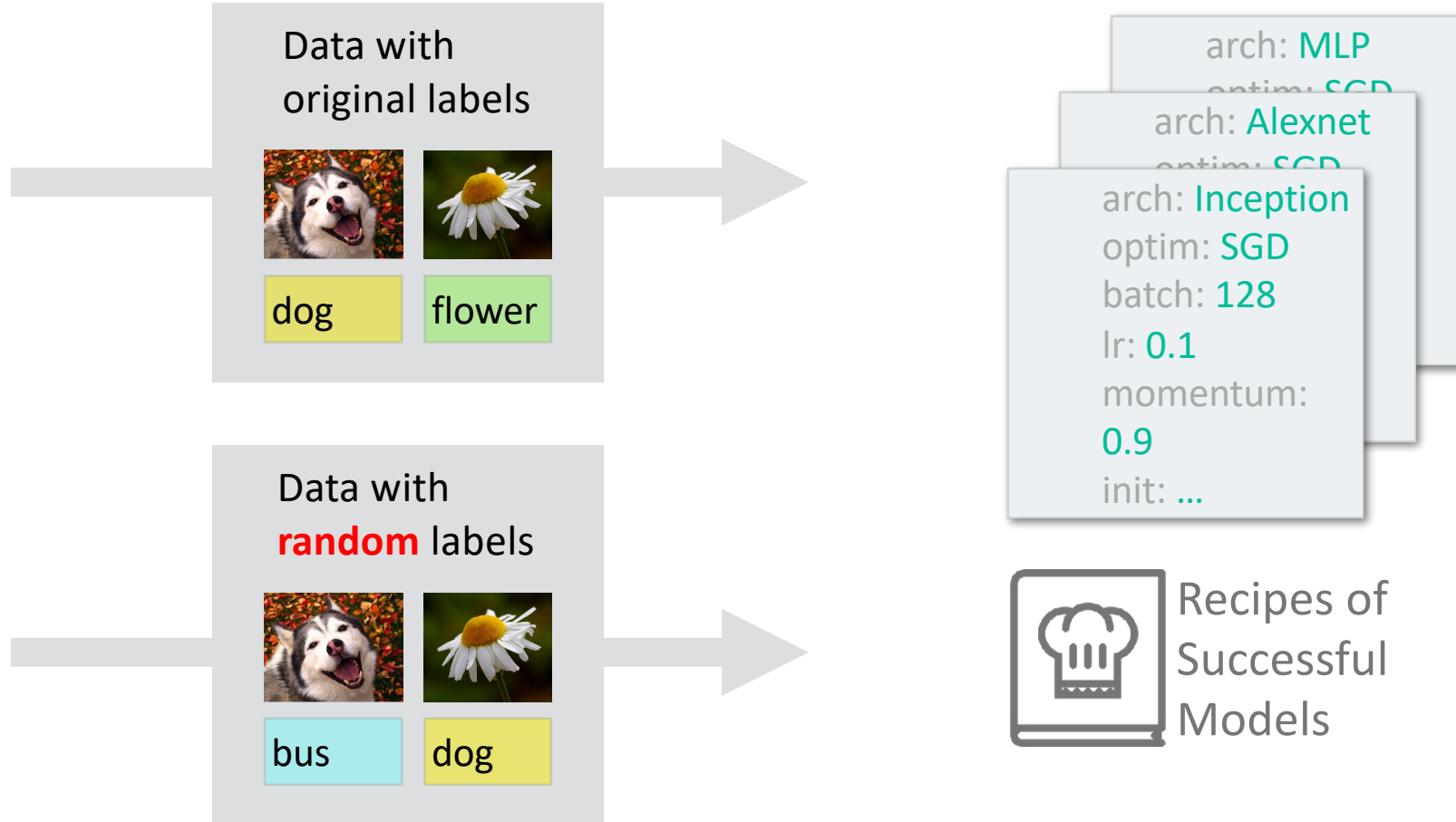
Flower

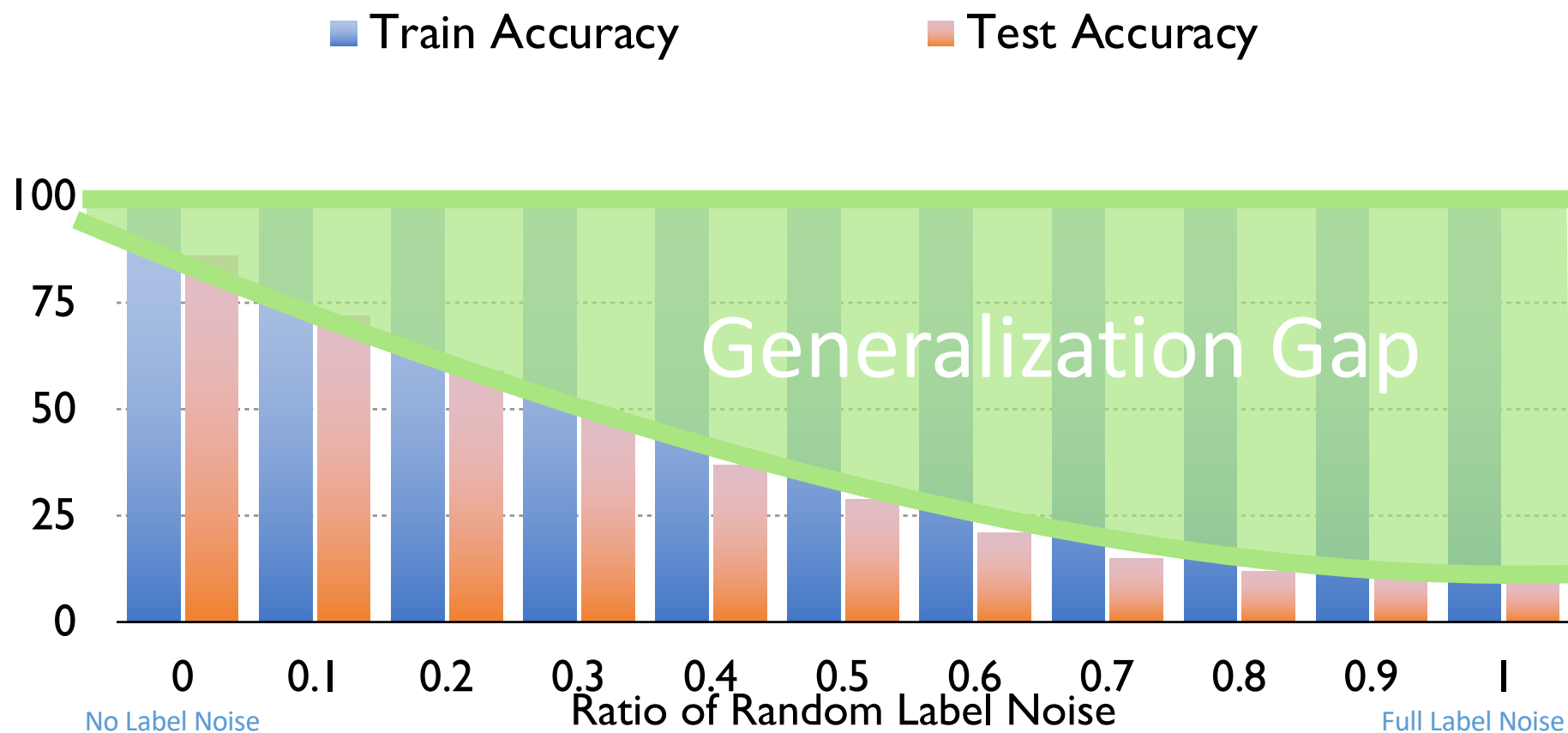


Bird

⋮

Randomization Test





Why is this amazing?

Networks are
able to fit
random labels



So they have huge
expressiveness,
SO WHAT?



When trained
normally, Networks
generalize!



Suresh Kumar Sharma with his certificates. He got into Limca Book of Records for memorizing 70,030 digits of the value of Pi

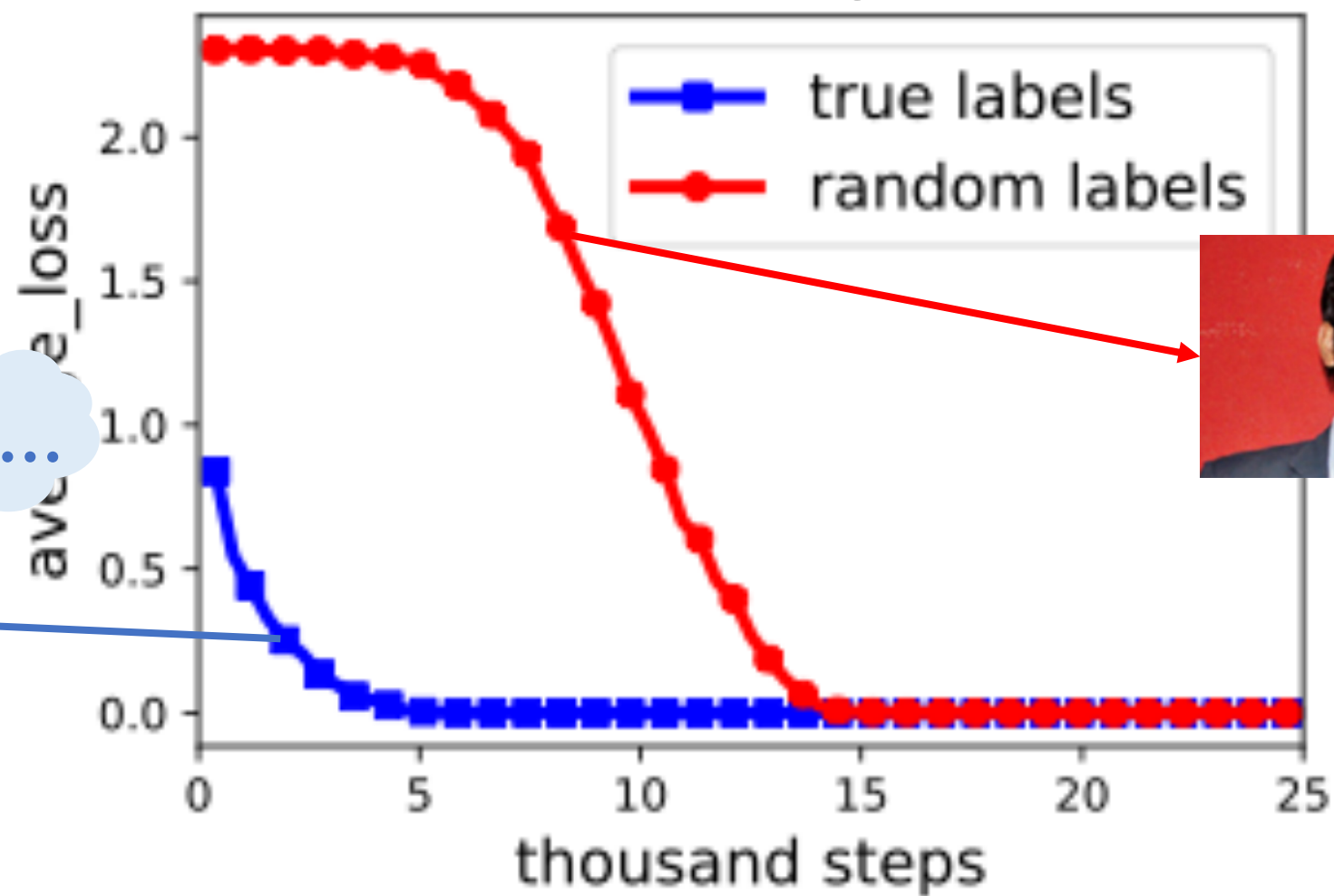
Q1: Can Suresh predict the 70031st digit?

**Q2: How long will it take Suresh to be able to say
1,2,3,4...70030?**

Will he then be able to predict the 70031st number?

Q3: Does Suresh use the same process in both cases?

Training



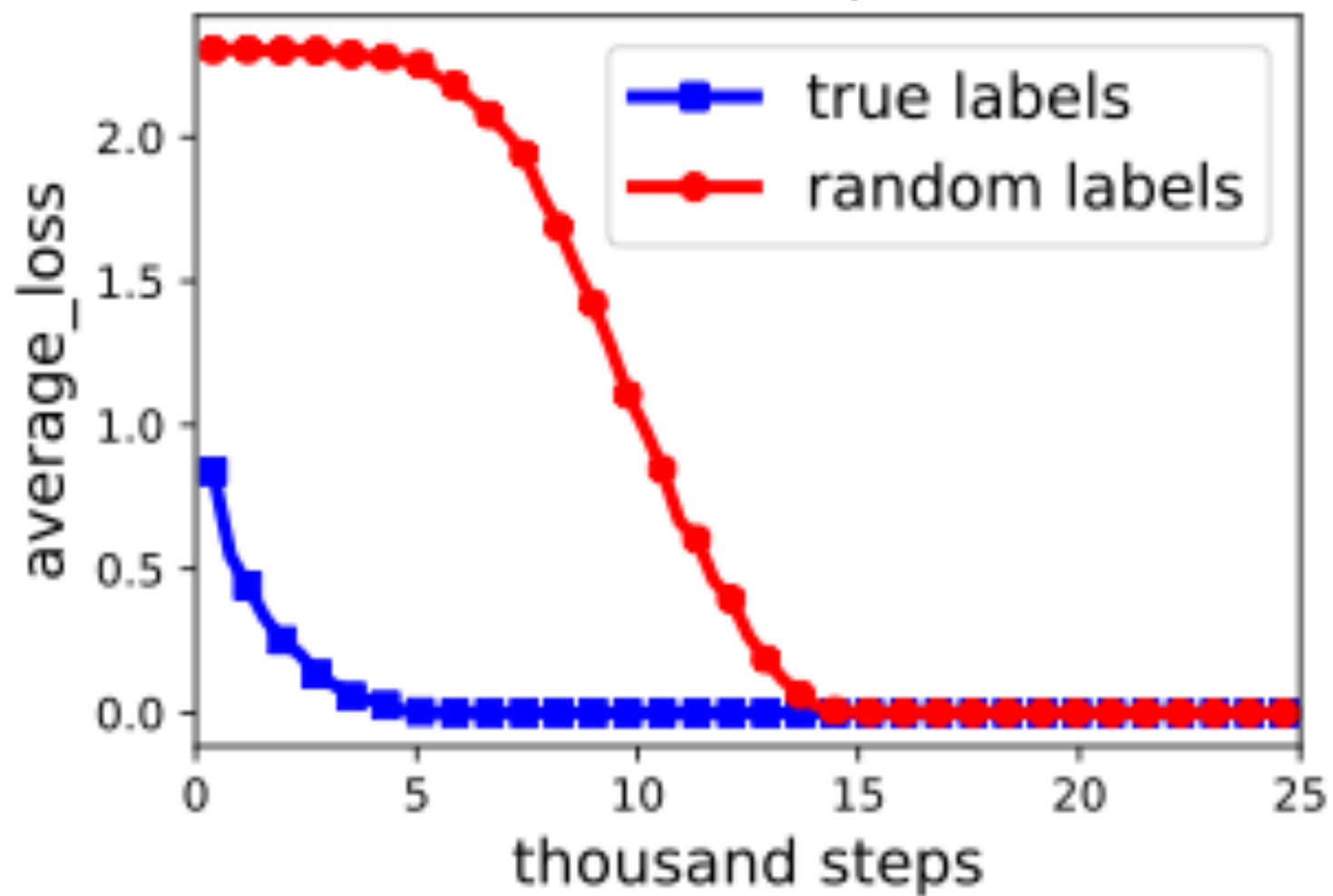
1,2,3,4...



π

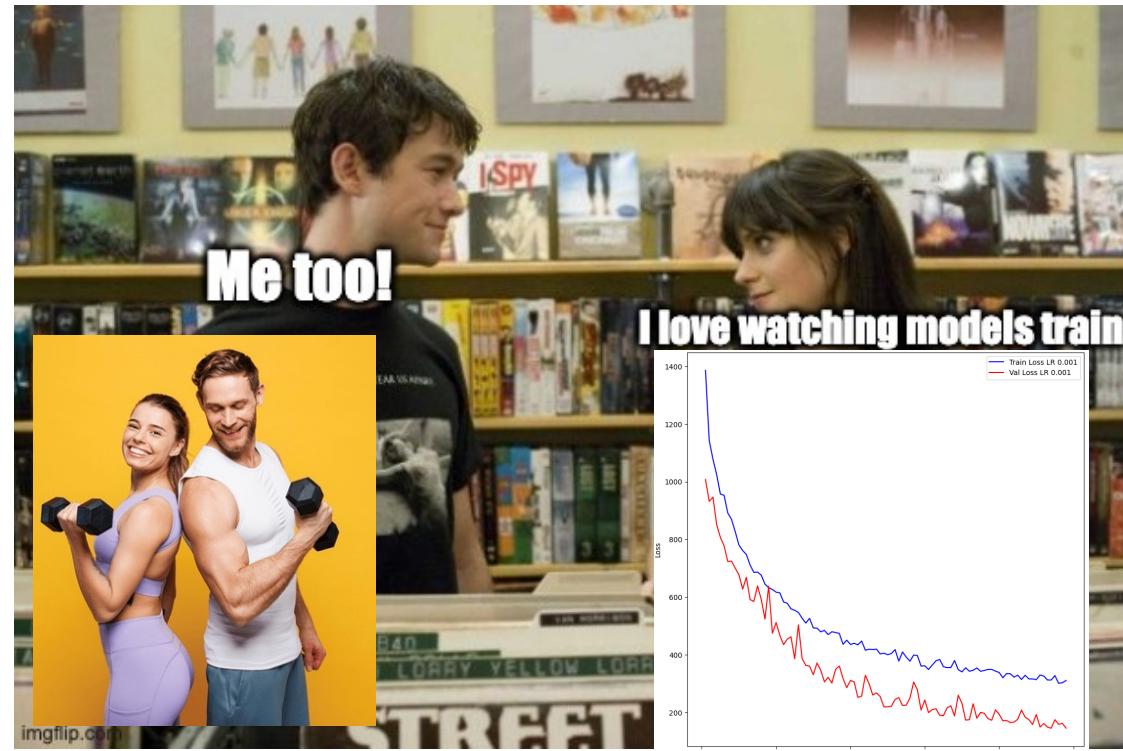


Training



<http://playground.tensorflow.org>

Thank you!



Next week's lecture:

Convolutional Neural Networks

